Dynamic Locomotion through Convex Optimisation

Complete Technical Documentation

Convex MPC Trot Controller for the Unitree Go2 Quadruped Robot

Flat-Ground Trotting · Stair Ascent & Descent · Terrain Adaptation

Based on: Di Carlo et al., "Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control", IROS 2018

Aniket Kumar Singh

March 6, 2026

Contents

1 Introduction and Project Overview **5**
1.1 Motivation . . . 5
1.2 Platform . . . 5
1.3 Feature Summary . . . 5
1.4 Repository Structure . . . 6

2 Robot Model and Physical Parameters **7**
2.1 Go2 Physical Parameters . . . 7
2.2 Body-Frame Inertia Tensor . . . 7
2.3 MPC State Vector . . . 7

3 Single Rigid Body Dynamics **9**
3.1 Centroidal Model Motivation . . . 9
3.2 Newton–Euler Equations . . . 9
3.2.1 Di Carlo Inertia Approximation . . . 9
3.3 Continuous-Time State-Space Model . . . 10
3.3.1 Euler-Angle Kinematics . . . 10
3.3.2 Position Kinematics . . . 10
3.3.3 Gravity Augmentation . . . 10
3.3.4 Full Continuous A Matrix . . . 10
3.3.5 Continuous B Matrix . . . 10
3.4 Zero-Order Hold Discretisation . . . 11

4 Trot Gait Scheduler **12**
4.1 Diagonal Trot Pattern . . . 12
4.2 Phase Computation . . . 12
4.3 Horizon Schedule . . . 12

5 State Estimation **13**
5.1 Ground-Truth Extraction . . . 13
5.2 Contact Detection Strategy . . . 13
5.2.1 Raw efc_force Reading . . . 13
5.2.2 Rolling-Maximum Buffer . . . 13
5.2.3 Height-Based Override (Critical Fix) . . . 13

6 Condensed Convex QP Formulation **15**
6.1 Prediction Horizon . . . 15
6.2 Condensed Dynamics . . . 15
6.2.1 Aqp Matrix . . . 15
6.2.2 Bqp Matrix . . . 15
6.3 Cost Function . . . 15
6.3.1 State Tracking . . . 15
6.3.2 Force Regularisation . . . 16

6.3.3 Full QP . . . 16
6.4 MPC Cost Weights . . . 16
6.4.1 Flat-Ground Weights . . . 16
6.4.2 Stair-Climbing Weights . . . 16
6.5 Friction Cone Constraints . . . 17
6.5.1 Physical Constraint . . . 17
6.5.2 Linearised Pyramid Approximation . . . 17
6.5.3 Terrain-Aware Rotation . . . 18
6.5.4 Swing-Leg Constraints . . . 18

7 Reference Trajectory Construction 19
7.1 Operator Commands . . . 19
7.2 Per-Step Reference . . . 19
7.3 Terrain-Aware Reference (Stairs) . . . 19
7.4 Height Integral Correction . . . 20

8 Swing Leg Controller 21
8.1 Minimum-Jerk Trajectory . . . 21
8.1.1 Height Bump . . . 21
8.1.2 Velocity and Acceleration . . . 21
8.2 Raibert Touchdown Heuristic . . . 22
8.3 Task-Space PD Controller . . . 22
8.4 GRF to Joint Torques (Stance) . . . 22
8.5 Actuator Reordering . . . 23

9 Controller State Machine 24
9.1 Three-Phase Architecture . . . 24
9.2 Phase A: PD Settling (0–1.5 s) . . . 24
9.2.1 Goal . . . 24
9.2.2 Controller . . . 24
9.2.3 Height Estimation . . . 24
9.3 Phase B: All-Stance MPC (1.5–3.5 s) . . . 25
9.3.1 Goal . . . 25
9.3.2 Contact Schedule . . . 25
9.3.3 GRF Initialisation . . . 25
9.3.4 Phase B Warmup (100 ms) . . . 25
9.4 Phase C: Trot MPC ($t > 3.5$ s) . . . 25
9.4.1 Entry Gate . . . 25
9.4.2 Phase C Warmup (300 ms) . . . 25
9.4.3 Normal Operation . . . 25
9.5 GRF Smoothing . . . 26
9.5.1 GRF Re-Seeding . . . 26
9.6 Collapse Detection and Recovery . . . 26

10 Flat-Ground Trotting: End-to-End Flow 27
10.1 Step-by-Step Control Loop . . . 27
10.2 Results for Flat-Ground Trotting . . . 28

11 Stair Climbing System 29
11.1 Stair Scene Geometry . . . 29
11.2 Terrain Mode Activation . . . 29
11.3 Terrain Height Map . . . 29
11.3.1 Discrete Step Function . . . 29
11.3.2 Smooth Ramp Approximation . . . 30
11.4 Adaptive CoM Height . . . 30

11.4.1 Target Height . . . 30
11.4.2 Asymmetric Rate Limiter . . . 30
11.5 Body Pitch Reference . . . 30
11.5.1 Geometric Derivation . . . 30
11.5.2 IIR Smoothing . . . 31
11.6 Per-Foot Stair Touchdown Placement . . . 31
11.6.1 Tread Snapping . . . 31
11.6.2 Height Map Touchdown . . . 31
11.7 Adaptive Swing Clearance . . . 31
11.8 Early Touchdown Detection . . . 31
11.9 Support Plane Estimation . . . 32
11.9.1 Purpose . . . 32
11.9.2 Least-Squares Plane Fit . . . 32
11.9.3 Support Frame Construction . . . 32
11.10 IIR Terrain Tracking . . . 32
11.11 Results for Stairs Climbing . . . 33

12 Complete Constraints Reference **34**

13 Module Dependency and Data Flow **35**
13.1 Module Dependency Graph . . . 35
13.2 Per-Step Data Flow (Phase C) . . . 35

14 Bug Fixes and Design Decisions **37**

15 Numerical Summary **39**

16 Conclusions **40**
16.1 Summary . . . 40
16.2 Limitations and Future Work . . . 40

List of Tables

2.1 Go2 physical parameters used in the MPC . . . 7
2.2 MPC state vector components . . . 8

6.1 Stair Q-weight tuning rationale (4 iterations of log analysis) . . . 17

11.1 Stair scene geometry . . . 29

12.1 All constraints active in the QP and elsewhere . . . 34

14.1 Complete bug fix and design decision log (19 items) . . . 37

15.1 Key numerical parameters . . . 39

Chapter 1

Introduction and Project Overview

1.1 Motivation

Legged robots must balance three competing requirements simultaneously: they must generate sufficient ground-reaction forces (GRFs) to support their weight, satisfy friction constraints at each foot, and coordinate leg swing trajectories that advance the body forward. Classical approaches couple these problems, making real-time control difficult.

This project implements the approach of Di Carlo et al. [1], which decouples stance-force optimisation from swing-foot placement by formulating the stance problem as a *convex* Quadratic Programme (QP) that can be solved on a 30 ms horizon in real time.

1.2 Platform

Hardware: Unitree Go2 quadruped robot.
Simulation: MuJoCo 3.x (via the `unitree_mujoco` scene).
Solver: `quadprog` (primary) / `osqp` (fallback).

1.3 Feature Summary

- Flat-ground diagonal trot at commanded forward velocity.
- Stair ascent and descent over a 5-step staircase (6 cm riser, 28 cm tread, 30 cm total height).
- Adaptive CoM height, body pitch reference, and swing clearance.
- Terrain-aware friction-cone constraints via support-plane estimation.
- Per-foot stair tread snapping and early-touchdown detection.
- Collapse detection and PD recovery mode.

1.4 Repository Structure

File	Role
`robot_params.py`	Physical parameters, Q weights, RobotState dataclass
`config.py`	MuJoCo IDs, hardware limits, debug flags
`dynamics.py`	SRB A/B matrices, ZOH discretisation
`gait_scheduler.py`	Trot gait contact schedule
`state_estimator.py`	Ground-truth reader, contact detection
`mpc_solver.py`	Condensed QP: condense, cost, constraints, solve
`swing_controller.py`	Minimum-jerk swing trajectory, task-space PD
`torque_utils.py`	GRF→torques, reference builder, PD stand, logger
`terrain_estimator.py`	Stair geometry, IIR terrain tracking
`support_plane.py`	SVD plane fit of stance feet
`mpc_controller.py`	Phase A/B/C state machine (top-level)
`simulation.py`	MuJoCo viewer + headless runner
`flat_trot.py`	Entry point — flat ground
`stairs_climb.py`	Entry point — stair scene

Chapter 2

Robot Model and Physical Parameters

2.1 Go2 Physical Parameters

The Unitree Go2 is modelled as a *Single Rigid Body* (SRB) at the centroidal level. Only the trunk mass and inertia enter the MPC; leg masses (approx. 1.5 kg/leg) are compensated by adding the joint-space gravity/-Coriolis bias $\boldsymbol{q}_{\text{frc_bias}}$ after the QP solve.

Table 2.1: Go2 physical parameters used in the MPC

Parameter	**Symbol**	**Value**
Total mass	m	$15.206\,\text{kg}$
Gravity	g	$9.81\,\text{m/s}^2$
Roll inertia (body)	I_{xx}	$0.10703\,\text{kg}\,\text{m}^2$
Pitch inertia (body)	I_{yy}	$0.09808\,\text{kg}\,\text{m}^2$
Yaw inertia (body)	I_{zz}	$0.02445\,\text{kg}\,\text{m}^2$
Hip ab/ad offset	l_{hip}	$0.08085\,\text{m}$
Thigh link length	l_{thigh}	$0.213\,\text{m}$
Calf link length	l_{calf}	$0.213\,\text{m}$
Hip origin FL (body frame)		$(+0.1934,\ +0.1125,\ 0)$
Hip origin FR (body frame)		$(+0.1934,\ -0.1125,\ 0)$
Hip origin RL (body frame)		$(-0.1934,\ +0.1125,\ 0)$
Hip origin RR (body frame)		$(-0.1934,\ -0.1125,\ 0)$

2.2 Body-Frame Inertia Tensor

$$\mathbf{I}_B = \text{diag}(I_{xx},\ I_{yy},\ I_{zz}) = \text{diag}(0.1070,\ 0.0981,\ 0.0245)\ \text{kg}\,\text{m}^2 \tag{2.1}$$

2.3 MPC State Vector

The 13-dimensional state vector used throughout the MPC is:

$$\boldsymbol{x} = [\phi\ \theta\ \psi\ p_x\ p_y\ p_z\ \omega_x\ \omega_y\ \omega_z\ v_x\ v_y\ v_z\ \ -g]^\top \in \mathbb{R}^{13} \tag{2.2}$$

Table 2.2: MPC state vector components

Index	**Symbol**	**Meaning**	**Unit**	**Frame**
0	ϕ	Roll (Euler ZYX)	rad	World
1	θ	Pitch (Euler ZYX)	rad	World
2	ψ	Yaw (Euler ZYX)	rad	World
3	p_x	CoM position x	m	World
4	p_y	CoM position y	m	World
5	p_z	CoM position z	m	World
6	ω_x	Angular velocity x	rad/s	World
7	ω_y	Angular velocity y	rad/s	World
8	ω_z	Angular velocity z	rad/s	World
9	v_x	Linear velocity x	m/s	World
10	v_y	Linear velocity y	m/s	World
11	v_z	Linear velocity z	m/s	World
12	$-g$	Gravity augmentation	m/s^2	—

The *gravity augmentation* trick appends the constant $-g$ as the 13th state so that gravity appears naturally in the linear dynamics via the $\mathbf{A}$ matrix without needing an affine term.

Chapter 3

Single Rigid Body Dynamics

3.1 Centroidal Model Motivation

A quadruped with $4 \times 3 = 12$ leg joints has 18 degrees of freedom (6 body + 12 joints). Real-time whole-body optimisation over a predictive horizon is intractable. The *single rigid body* (SRB) model reduces the problem to 13 states and $4 \times 3 = 12$ inputs (one 3-D force vector per foot), giving a convex QP solvable in $< 5\,\text{ms}$.

The SRB assumptions are:

1. Leg masses are small compared to the trunk ($\approx 10\%$ each).
2. Foot accelerations are small (slow enough swings).
3. The inertia tensor can be approximated by rotating the body-frame tensor by yaw only (Di Carlo approximation).

3.2 Newton–Euler Equations

For a rigid body with n_c foot contacts, Newton's second law gives:

$$m\,\dot{\boldsymbol{v}} = \sum_{i=1}^{n_c} \boldsymbol{f}_i + m\,\boldsymbol{g} \tag{3.1}$$

$$\hat{\mathbf{I}}\,\dot{\boldsymbol{\omega}} = \sum_{i=1}^{n_c} \boldsymbol{r}_i \times \boldsymbol{f}_i \tag{3.2}$$

where $\boldsymbol{f}_i \in \mathbb{R}^3$ is the ground-reaction force (GRF) at foot i, $\boldsymbol{r}_i = \boldsymbol{p}_i - \boldsymbol{p}_{\text{CoM}}$ is the moment arm from the CoM to foot i, and $\hat{\mathbf{I}}$ is the world-frame inertia tensor.

3.2.1 Di Carlo Inertia Approximation

Computing the full world-frame inertia $\mathbf{R}\,\mathbf{I}_B\,\mathbf{R}^\top$ at every time step introduces trilinear products that destroy convexity. Di Carlo et al. approximate this by rotating only around yaw:

$$\hat{\mathbf{I}}(\psi) = \mathbf{R}_z(\psi)\;\mathbf{I}_B\;\mathbf{R}_z(\psi)^\top \tag{3.3}$$

where $\mathbf{R}_z(\psi)$ is the rotation matrix for yaw angle ψ. This is exact on flat ground when roll and pitch are small, which is a reasonable approximation during trotting locomotion.

3.3 Continuous-Time State-Space Model

3.3.1 Euler-Angle Kinematics

Using the small-angle Euler-rate approximation from Eq. (12) of Di Carlo:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} \approx \mathbf{R}_z(\psi)^\top \, \boldsymbol{\omega} \tag{3.4}$$

This gives the upper-left block of $\mathbf{A}_c$:

$$\mathbf{A}_c[0{:}3,\ 6{:}9] = \mathbf{R}_z(\psi)^\top \tag{3.5}$$

3.3.2 Position Kinematics

Linear velocity integrates directly into position:

$$\mathbf{A}_c[3{:}6,\ 9{:}12] = \mathbf{I}_3 \tag{3.6}$$

3.3.3 Gravity Augmentation

The 13th state $x_{12} = -g$ feeds into $\dot{v}_z$:

$$\mathbf{A}_c[11,\ 12] = 1 \quad \Longrightarrow \quad \dot{v}_z \supseteq x_{12} = -g \tag{3.7}$$

3.3.4 Full Continuous A Matrix

$$\mathbf{A}_c(\psi) = \begin{pmatrix} \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \mathbf{R}_z(\psi)^\top & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times1} \\ \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \mathbf{I}_3 & \mathbf{0}_{3\times1} \\ \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times1} \\ \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} & \boldsymbol{e}_3 \\ \mathbf{0}_{1\times3} & \mathbf{0}_{1\times3} & \mathbf{0}_{1\times3} & \mathbf{0}_{1\times3} & 0 \end{pmatrix} \in \mathbb{R}^{13\times13} \tag{3.8}$$

where $\boldsymbol{e}_3 = [0, 0, 1]^\top$ places the gravity constant in the v_z equation.

3.3.5 Continuous B Matrix

The $\mathbf{B}_c$ matrix maps the control input $\boldsymbol{u} = [\boldsymbol{f}_1^\top, \boldsymbol{f}_2^\top, \boldsymbol{f}_3^\top, \boldsymbol{f}_4^\top]^\top \in \mathbb{R}^{12}$ to state derivatives. For each stance foot i ($c_i = 1$):

$$\mathbf{B}_c[6{:}9,\ 3i{:}3(i+1)] = \hat{\mathbf{I}}^{-1}\,[\boldsymbol{r}_i]_\times \qquad \text{(angular dynamics)} \tag{3.9}$$

$$\mathbf{B}_c[9{:}12,\ 3i{:}3(i+1)] = \frac{\mathbf{I}_3}{m} \qquad \text{(linear dynamics)} \tag{3.10}$$

where $[\boldsymbol{r}]_\times$ denotes the 3×3 skew-symmetric matrix (cross-product operator) of vector $\boldsymbol{r}$:

$$[\boldsymbol{r}]_\times = \begin{pmatrix} 0 & -r_z & r_y \\ r_z & 0 & -r_x \\ -r_y & r_x & 0 \end{pmatrix} \tag{3.11}$$

Swing legs ($c_i = 0$) contribute zero columns to $\mathbf{B}_c$. If all contacts are zero (fallback), all feet are treated as stance.

Critical sign convention: The GRF $\boldsymbol{f}_i$ is the force the *ground exerts on the foot* (positive z = upward). The skew cross-product $\boldsymbol{r}_i \times \boldsymbol{f}_i$ already has the correct sign for Newton-Euler Eq. (3.2).

Frame convention: Moment arms $\boldsymbol{r}_i$ are kept in the *world frame*. The $\mathbf{B}$ matrix maps world-frame forces to world-frame accelerations; mixing frames (e.g. rotating $\boldsymbol{r}$ into the support plane while $\boldsymbol{f}$ remains in world frame) produces a physically wrong cross product. The support-plane rotation is used *only* in the friction-cone constraints.

3.4 Zero-Order Hold Discretisation

Given the continuous pair $(\mathbf{A}_c, \mathbf{B}_c)$ and a timestep Δt, the ZOH discrete matrices are obtained from the matrix exponential:

$$\begin{pmatrix} \mathbf{A}_d \\ \mathbf{B}_d \end{pmatrix} = \exp\left(\begin{pmatrix} \mathbf{A}_c & \mathbf{B}_c \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \Delta t \right)_{:n_s,\,:} \tag{3.12}$$

In practice, we form the $(n_s{+}n_u){\times}(n_s{+}n_u)$ matrix $\mathbf{M} = [\mathbf{A}_c, \mathbf{B}_c;\ \mathbf{0}, \mathbf{0}]\,\Delta t$, compute $e^{\mathbf{M}}$ via `scipy.linalg.expm`, and read off:

$$\mathbf{A}_d = e^{\mathbf{M}}[0{:}n_s,\ 0{:}n_s], \qquad \mathbf{B}_d = e^{\mathbf{M}}[0{:}n_s,\ n_s{:}] \tag{3.13}$$

This is re-computed at *every* MPC solve, because $\mathbf{B}_c$ changes with foot positions and the contact schedule, making the system *time-varying* over the horizon.

Chapter 4

Trot Gait Scheduler

4.1 Diagonal Trot Pattern

A trot gait alternates two *diagonal pairs*:

- Pair 1: FL (front-left) + RR (rear-right)
- Pair 2: FR (front-right) + RL (rear-left)

Parameters:

$$T = 0.40\,\text{s} \qquad \text{(full gait cycle period, 2.5 Hz)}$$
$$d = 0.50 \qquad \text{(duty cycle: 50\% stance, 50\% swing)}$$
$$T_{\text{stance}} = 0.20\,\text{s}, \quad T_{\text{swing}} = 0.20\,\text{s}$$

4.2 Phase Computation

The phase offsets for each leg are:

$$\text{offset} = [0.0,\ 0.5,\ 0.5,\ 0.0] \quad \text{for FL, FR, RL, RR respectively} \tag{4.1}$$

Phase of leg i at simulation time t:

$$\phi_i(t) = \left(\frac{t - t_0}{T} + \text{offset}_i\right) \bmod 1 \tag{4.2}$$

where t_0 is the gait *activation time* (time when Phase C begins). Leg i is in stance when $\phi_i < d$, swing otherwise.

Why relative phase matters (Bug Fix): If $t_0 = 0$ and the simulation has been running for $t = 4.5\,\text{s}$, the phase $4.5/0.4 = 11.25 \Rightarrow \phi = 0.25$ places the robot mid-cycle, causing the first swing step to be incorrectly timed. By anchoring to t_0 at activation time, the gait always starts cleanly at $\phi = 0$.

4.3 Horizon Schedule

For the MPC, a $(K \times 4)$ boolean schedule matrix is precomputed:

$$\mathbf{S}[k, i] = \mathbf{1}[\phi_i(t_0^{\text{mpc}} + k\,\Delta t) < d] \quad k = 0, \ldots, K-1 \tag{4.3}$$

where t_0^{mpc} is the current simulation time at each MPC call.

Chapter 5

State Estimation

5.1 Ground-Truth Extraction

Since this is a simulation-based controller, the state estimator reads ground-truth values directly from MuJoCo data buffers every physics step (2 ms at 500 Hz):

- $\boldsymbol{p} = \texttt{d.qpos}[0{:}3]$ — CoM position.
- $\hat{\boldsymbol{q}} = \texttt{d.qpos}[3{:}7]$ (wxyz) — body quaternion, converted to rotation matrix $\mathbf{R}$ and Euler angles $[\phi, \theta, \psi]$ via ZYX convention.
- $\boldsymbol{v} = \texttt{d.qvel}[0{:}3]$ — CoM linear velocity (world).
- $\boldsymbol{\omega} = \texttt{d.qvel}[3{:}6]$ — angular velocity (world).
- Joint positions $\boldsymbol{q}$ and velocities $\dot{\boldsymbol{q}}$ read from their respective qpos/qvel slices.
- Foot world positions from `d.xpos[FOOT_BODY_IDS[leg]]`.

5.2 Contact Detection Strategy

5.2.1 Raw efc_force Reading

MuJoCo reports constraint forces via `d.efc_force[addr]`. For each active contact, the geom IDs are checked against a runtime-built map (constructed from `mj_name2id` lookups), and the force is accumulated per leg:

$$c_{f,i}^{\text{raw}} = \sum_{\text{contacts with foot } i} |\texttt{efc_force}[\texttt{addr}]| \tag{5.1}$$

5.2.2 Rolling-Maximum Buffer

A 10-step (20 ms) rolling-maximum buffer suppresses single-step dropouts:

$$c_{f,i} = \max_{j \in \{0,\ldots,9\}} c_{f,i}^{\text{raw}}[t - j\,\Delta t_{\text{phys}}] \tag{5.2}$$

5.2.3 Height-Based Override (Critical Fix)

MuJoCo's contact solver re-initialises its warm-start when the actuator command vector changes abruptly, causing `efc_force` to drop to *exactly zero* for up to 300 ms even when all feet are physically on the ground. A rolling buffer of any practical size cannot bridge this.

The correct fix uses the body height p_z as a proxy:

$$\text{contact}_i = \begin{cases} \texttt{True} & \text{if } p_z > p_{z,\text{collapse}} = 0.20\,\text{m} \\ c_{f,i} > \tau_{\text{cf}} & \text{otherwise} \end{cases} \tag{5.3}$$

with $\tau_{\mathrm{cf}} = 10\,\mathrm{N}$. When $p_z > 0.20\,\mathrm{m}$ the robot is clearly standing; forcing all contacts true is always correct because:

1. Phase B uses all-stance regardless of contact state.
2. Phase C uses the *gait scheduler* for contact decisions, not this estimator.
3. Collapse detection uses p_z itself, so a false "not in contact" cannot trigger it.

Chapter 6

Condensed Convex QP Formulation

6.1 Prediction Horizon

With MPC timestep $\Delta t_{\text{mpc}} = 30\,\text{ms}$ and horizon $K = 10$ steps, the controller looks 300 ms ahead at each solve.

6.2 Condensed Dynamics

Define the stacked state trajectory $\mathbf{X} \in \mathbb{R}^{Kn_s}$ and stacked input trajectory $\mathbf{U} \in \mathbb{R}^{Kn_u}$. The time-varying discrete system $\boldsymbol{x}_{k+1} = \mathbf{A}_k \boldsymbol{x}_k + \mathbf{B}_k \boldsymbol{u}_k$ gives the *condensed* form:

$$\mathbf{X} = \mathbf{A}_{\text{qp}}\, \boldsymbol{x}_0 + \mathbf{B}_{\text{qp}}\, \mathbf{U} \tag{6.1}$$

6.2.1 Aqp Matrix

$\mathbf{A}_{\text{qp}} \in \mathbb{R}^{Kn_s \times n_s}$ stacks the state-transition products:

$$\mathbf{A}_{\text{qp}} = \begin{pmatrix} \mathbf{A}_0 \\ \mathbf{A}_1 \mathbf{A}_0 \\ \vdots \\ \mathbf{A}_{K-1} \cdots \mathbf{A}_0 \end{pmatrix} \quad \text{where} \quad \Phi_k = \prod_{j=0}^{k} \mathbf{A}_j \tag{6.2}$$

6.2.2 Bqp Matrix

$\mathbf{B}_{\text{qp}} \in \mathbb{R}^{Kn_s \times Kn_u}$ is lower-block-triangular (causal):

$$\mathbf{B}_{\text{qp}}[r, c] = \begin{cases} \Phi_{r,c}\, \mathbf{B}_c & \text{if } r \geq c \\ \mathbf{0} & \text{otherwise} \end{cases} \quad \text{where} \quad \Phi_{r,c} = \prod_{j=c+1}^{r} \mathbf{A}_j \tag{6.3}$$

6.3 Cost Function

6.3.1 State Tracking

Define the diagonal weight matrix $\mathbf{L} = \mathbf{I}_K \otimes \text{diag}(\boldsymbol{q})$ where $\boldsymbol{q} \in \mathbb{R}^{13}$ are the state weights. The state tracking cost is:

$$J_{\text{state}} = \|\mathbf{X} - \mathbf{X}_{\text{ref}}\|_{\mathbf{L}}^2 = (\mathbf{A}_{\text{qp}} \boldsymbol{x}_0 + \mathbf{B}_{\text{qp}} \mathbf{U} - \mathbf{X}_{\text{ref}})^\top \mathbf{L} (\mathbf{A}_{\text{qp}} \boldsymbol{x}_0 + \mathbf{B}_{\text{qp}} \mathbf{U} - \mathbf{X}_{\text{ref}}) \tag{6.4}$$

6.3.2 Force Regularisation

To prevent unrealistically large forces and ensure uniqueness when multiple solutions exist:

$$J_{\text{force}} = \alpha \, \|\mathbf{U}\|_2^2, \qquad \alpha = 3 \times 10^{-4} \tag{6.5}$$

6.3.3 Full QP

$$\min_{\mathbf{U}} \quad \frac{1}{2} \mathbf{U}^\top \mathbf{H} \, \mathbf{U} + \boldsymbol{g}^\top \mathbf{U} \tag{6.6}$$

$$\mathbf{H} = 2 \left(\mathbf{B}_{\text{qp}}^\top \mathbf{L} \, \mathbf{B}_{\text{qp}} + \alpha \, \mathbf{I}_{Kn_u} \right) \tag{6.7}$$

$$\boldsymbol{g} = 2 \, \mathbf{B}_{\text{qp}}^\top \mathbf{L} \left(\mathbf{A}_{\text{qp}} \, \boldsymbol{x}_0 - \mathbf{X}_{\text{ref}} \right) \tag{6.8}$$

6.4 MPC Cost Weights

6.4.1 Flat-Ground Weights

$$\boldsymbol{q}_{\text{flat}} = \text{diag}(10,\ 20,\ 1,\ \ 1,\ 1,\ 150,\ \ 1,\ 5,\ 1,\ \ 2,\ 2,\ 5,\ \ 0) \tag{6.9}$$

$$\text{for } [\phi, \theta, \psi,\ p_x, p_y, p_z,\ \omega_x, \omega_y, \omega_z,\ v_x, v_y, v_z,\ -g]$$

The high $p_z = 150$ strongly penalises height deviation, keeping the CoM at the settled height. The pitch weight $\theta = 20$ prevents forward/backward tipping during trot.

6.4.2 Stair-Climbing Weights

$$\boldsymbol{q}_{\text{stairs}} = \text{diag}(60,\ 120,\ 1,\ \ 1,\ 15,\ 100,\ \ 8,\ 15,\ 1,\ \ 25,\ 15,\ 8,\ \ 0) \tag{6.10}$$

The rationale for each change from the flat weights:

Table 6.1: Stair Q-weight tuning rationale (4 iterations of log analysis)

State	Flat	Stairs	Rationale
ϕ (roll)	10	60	Roll is the primary failure mode. Diagonal stance (FL on step, RR on flat) creates ≈9.3 Nm roll torque. Equal priority with p_z.
θ (pitch)	20	120	Pitch diverges on stairs. Weight *dominant* over p_z (2:1) prevents height-tracking force from creating nose-up moment.
p_y	1	15	Zero lateral authority caused v_y to explode to 0.38 m/s on first step. Strong lateral position tracking required.
p_z	150	100	Reduced from flat (height tracking slightly sacrificed for pitch priority). Still 67% of flat strength.
ω_x	1	8	Roll rate reached 15°/s before divergence; strong damping needed.
ω_y	5	15	Pitch rate reached 10°/s sustained; penalty 15×0.17^2=0.44 provides meaningful deceleration.
v_x	2	25	Robot surged 0.05→0.21 m/s (4× overspeed), leaving <1 s per tread. Strong forward-speed regulation.
v_y	2	15	Lateral velocity regulation to complement p_y tracking.
v_z	5	8	Vertical velocity oscillation (±0.15 m/s) couples into pitch.

6.5 Friction Cone Constraints

6.5.1 Physical Constraint

The foot can only push, not pull, and must stay within the friction cone:

$$f_{n,i} \geq 0, \qquad \sqrt{f_{t,i}^2} \leq \mu\, f_{n,i} \tag{6.11}$$

where $f_{n,i}$ is the normal force and $f_{t,i}$ is the tangential force, $\mu = 0.7$ is the friction coefficient.

6.5.2 Linearised Pyramid Approximation

The circular cone is replaced by a square pyramid (4 planes):

$$f_{n,i} \geq f_{\min} = 5\,\mathrm{N} \quad \text{(positive normal, min support force)} \tag{6.12}$$

$$f_{n,i} \leq f_{\max} = 200\,\mathrm{N} \quad \text{(leg load limit)} \tag{6.13}$$

$$f_{t_x,i} - \mu\, f_{n,i} \leq 0 \tag{6.14}$$

$$-f_{t_x,i} - \mu\, f_{n,i} \leq 0 \tag{6.15}$$

$$f_{t_y,i} - \mu\, f_{n,i} \leq 0 \tag{6.16}$$

$$-f_{t_y,i} - \mu\, f_{n,i} \leq 0 \tag{6.17}$$

Equations (6.14)–(6.17) give 5 constraints per stance leg (1 normal bound + 4 tangential planes).

6.5.3 Terrain-Aware Rotation

On sloped terrain (stairs), the support-plane normal $\hat{\boldsymbol{n}}$ deviates from $[0, 0, 1]^\top$. The friction cone is rotated into the support frame:

$$\boldsymbol{f}_{\text{plane}} = \mathbf{R}_{\text{plane}}^\top \boldsymbol{f}_{\text{world}} \tag{6.18}$$

where $\mathbf{R}_{\text{plane}}$ is the orthonormal basis of the support plane (columns: $\hat{\boldsymbol{x}}_{\text{plane}}$, $\hat{\boldsymbol{y}}_{\text{plane}}$, $\hat{\boldsymbol{n}}$). The normal constraint then becomes:

$$\hat{\boldsymbol{n}}^\top \boldsymbol{f}_i \in [f_{\min},\ f_{\max}] \tag{6.19}$$

and the tangential constraints read:

$$\hat{\boldsymbol{t}}_{x/y}^\top \boldsymbol{f}_i - \mu\, \hat{\boldsymbol{n}}^\top \boldsymbol{f}_i \leq 0 \tag{6.20}$$

6.5.4 Swing-Leg Constraints

For each swing leg i ($S[k, i] = 0$), all three force components are zeroed:

$$f_{x,i} = f_{y,i} = f_{z,i} = 0 \quad \text{(swing, 3 equality constraints)} \tag{6.21}$$

Chapter 7

Reference Trajectory Construction

7.1 Operator Commands

The operator specifies body-frame commands:

$$\boldsymbol{c} = [v_x^{\text{cmd}},\ v_y^{\text{cmd}},\ \omega_z^{\text{cmd}}] \tag{7.1}$$

7.2 Per-Step Reference

For each horizon step k at projected position $x_k = p_x + v_x^{\text{cmd}}\, k\, \Delta t$:

$$\boldsymbol{r}_k = \begin{pmatrix} 0 \\ \theta_{\text{ref}} \\ \psi_0 + \omega_z^{\text{cmd}}\, k\, \Delta t \\ x_k \\ p_y + v_y^{\text{cmd}}\, k\, \Delta t \\ p_{z,\text{des}}^{(k)} \\ 0 \\ 0 \\ \omega_z^{\text{cmd}} \\ v_x^{\text{cmd}} \\ v_y^{\text{cmd}} \\ 0 \\ -g \end{pmatrix} \tag{7.2}$$

7.3 Terrain-Aware Reference (Stairs)

When terrain mode is active, both θ_{ref} and $p_{z,\text{des}}$ are computed as a *delta from the current projected position*:

$$\theta_{\text{ref}}^{(k)} = \theta_{\text{ref,smooth}} + \Delta\theta(x_k) \tag{7.3}$$

$$\Delta\theta(x_k) = \texttt{pitch_ref}(x_k) - \texttt{pitch_ref}(x_0) \tag{7.4}$$

$$p_{z,\text{des}}^{(k)} = p_{z,\text{des,smooth}} + \Delta p_z(x_k) \tag{7.5}$$

$$\Delta p_z(x_k) = \texttt{target_pz}(x_k, \ell) - \texttt{target_pz}(x_0, \ell) \tag{7.6}$$

where ℓ is the nominal CoM clearance above terrain. This delta formulation preserves the IIR-smoothed current values at $k = 0$ while anticipating terrain changes in future horizon steps.

7.4 Height Integral Correction

A small integral correction prevents steady-state height error:

$$p_{z,\text{corrected}} = p_{z,\text{des}} + \text{clip}(0.1\, e_{z,\text{int}},\ -0.05,\ 0.05) \tag{7.7}$$

Chapter 8

Swing Leg Controller

8.1 Minimum-Jerk Trajectory

The swing trajectory interpolates from the lift-off position $\boldsymbol{p}_0$ to the touchdown position $\boldsymbol{p}_f$ using a *minimum-jerk* polynomial:

$$s(t) = \frac{t - t_{\text{liftoff}}}{T_{\text{swing}}}, \quad s \in [0, 1] \tag{8.1}$$

$$P_{\text{mj}}(s) = 10s^3 - 15s^4 + 6s^5 \tag{8.2}$$

The position trajectory is:

$$\boldsymbol{p}_{\text{des}}(s) = \boldsymbol{p}_0 + (\boldsymbol{p}_f - \boldsymbol{p}_0)\, P_{\text{mj}}(s) + z_{\text{bump}}(s)\, \hat{\boldsymbol{e}}_z \tag{8.3}$$

8.1.1 Height Bump

A smooth height bump ensures foot clearance above the terrain:

$$z_{\text{bump}}(s) = h_{\text{sw}} \cdot 64\, s^3 (1 - s)^3 \tag{8.4}$$

Properties:

- $z_{\text{bump}}(0) = z_{\text{bump}}(1) = 0$ (no displacement at endpoints).
- Peak at $s = 0.5$: $z_{\text{bump}}(0.5) = h_{\text{sw}}$.
- $\dot{z}_{\text{bump}}(0) = \dot{z}_{\text{bump}}(1) = 0$ (smooth lift/land).

For flat ground: $h_{\text{sw}} = 0.08\,\text{m}$. On stairs: h_{sw} is terrain-adaptive (see Chapter 11).

8.1.2 Velocity and Acceleration

$$\dot{P}_{\text{mj}}(s) = 30s^2 - 60s^3 + 30s^4 \tag{8.5}$$

$$\ddot{P}_{\text{mj}}(s) = 60s - 180s^2 + 120s^3 \tag{8.6}$$

Foot velocity: $\boldsymbol{v}_{\text{des}} = (\boldsymbol{p}_f - \boldsymbol{p}_0)\, \dot{P}_{\text{mj}} / T_{\text{swing}}$

8.2 Raibert Touchdown Heuristic

The foot touchdown position is determined by the Raibert heuristic [2]:

$$\boldsymbol{p}_{\text{td}} = \boldsymbol{p}_{\text{hip}} + \underbrace{\boldsymbol{v}_{\text{cmd}}\, t_{\text{pred}}}_{\text{nominal drift}} + \underbrace{k_v\,(\boldsymbol{v}_{\text{actual}} - \boldsymbol{v}_{\text{cmd}})}_{\text{error correction}} \tag{8.7}$$

where:

$$t_{\text{pred}} = \frac{T}{2} = \frac{T_{\text{swing}} + 0.5\,T_{\text{stance}}}{2} \approx 0.20\,\text{s} \tag{8.8}$$

$$k_{v,x} = 0.4\,T \approx 0.16\,\text{m}\,\text{s} \tag{8.9}$$

$$k_{v,y} = 0.05\,T \approx 0.02\,\text{m}\,\text{s} \quad \text{(reduced to prevent lateral instability)} \tag{8.10}$$

Command frame: The commanded velocity is in the body frame. It is rotated to the world frame before applying the Raibert formula:

$$\begin{pmatrix} v_x^w \\ v_y^w \end{pmatrix} = \begin{pmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{pmatrix} \begin{pmatrix} v_x^{\text{cmd}} \\ v_y^{\text{cmd}} \end{pmatrix} \tag{8.11}$$

Overcorrection fix: The original implementation used $\boldsymbol{p}_{\text{td}} = \boldsymbol{p}_{\text{hip}} + \boldsymbol{v}_{\text{actual}}\,(t_{\text{pred}} + k_v)$, which applies $(t_{\text{pred}} + k_v)\,v_{\text{actual}}$ as one block. At a 0.51 m/s lateral drift with $k_v = 0.08$, this gives a 3.5× larger correction than intended. The corrected formulation separates nominal drift (proportional to *commanded* velocity) from error correction (proportional to velocity *error*).

8.3 Task-Space PD Controller

For each swing leg, the joint torques are computed as:

$$\boldsymbol{\tau}_{\text{swing}} = \mathbf{J}_i^\top \underbrace{\left[\mathbf{K}_p\,(\boldsymbol{p}_{\text{des}} - \boldsymbol{p}_{\text{cur}}) + \mathbf{K}_d\,(\boldsymbol{v}_{\text{des}} - \boldsymbol{v}_{\text{cur}})\right]}_{\boldsymbol{F}_{\text{task}}} + \boldsymbol{\tau}_{\text{bias}} \tag{8.12}$$

where:

$$\mathbf{K}_p = \text{diag}(400, 400, 400)\,\text{N/m} \tag{8.13}$$

$$\mathbf{K}_d = \text{diag}(75, 75, 75)\,\text{N}\,\text{s/m} \tag{8.14}$$

$$\mathbf{J}_i = 3{\times}3 \text{ foot Jacobian (leg DOFs)} \tag{8.15}$$

$$\boldsymbol{\tau}_{\text{bias}} = \texttt{qfrc_bias}[\text{leg joints}] \quad \text{(gravity/Coriolis compensation)} \tag{8.16}$$

Foot velocity: The full foot velocity uses the complete $(3 \times n_v)$ Jacobian (body + all leg DOFs):

$$\boldsymbol{v}_{\text{cur}} = \mathbf{J}_{\text{full}}\,\dot{\boldsymbol{q}}_{\text{full}} \tag{8.17}$$

This includes body translational/angular velocity, which is significant when the body is rolling or pitching.

8.4 GRF to Joint Torques (Stance)

For stance legs, the MPC ground-reaction forces are converted to joint torques:

$$\boldsymbol{\tau}_{\text{stance,i}} = -\mathbf{J}_i^\top\,\boldsymbol{f}_i + \boldsymbol{\tau}_{\text{bias,i}} \tag{8.18}$$

Sign convention (critical): The MPC gives $\boldsymbol{f}_i$ as the force the *ground exerts on the foot* (upward positive). To generate this upward reaction, the joint must push *down* against the ground. For a knee joint with $J_z < 0$ at nominal stance, $-\mathbf{J}^\top\boldsymbol{f}$ gives extension torque (pushing down), which is physically correct.

8.5 Actuator Reordering

MuJoCo's actuator order for the Go2 differs from the internal leg ordering:

Internal order	**Indices**	**MuJoCo ctrl indices**
FL	0–2	ctrl[3–5]
FR	3–5	ctrl[0–2]
RL	6–8	ctrl[9–11]
RR	9–11	ctrl[6–8]

The `leg_to_ctrl()` function performs this remapping before writing to `data.ctrl`.

Chapter 9

Controller State Machine

9.1 Three-Phase Architecture

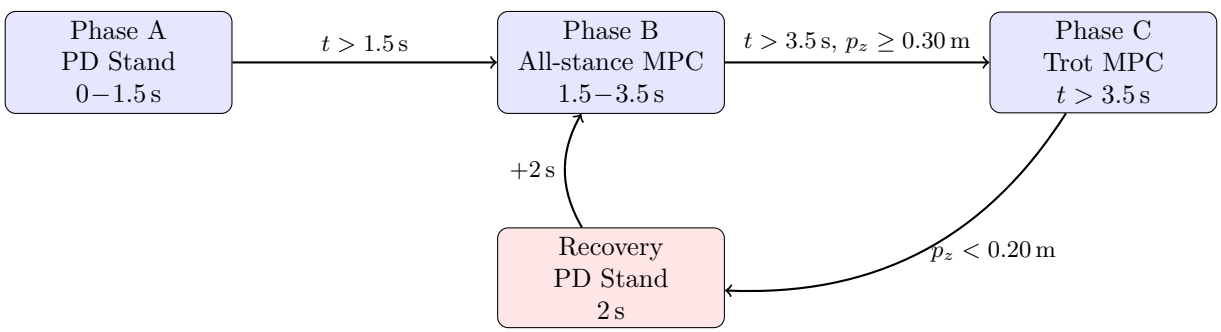


Figure 9.1: MPC controller three-phase state machine

9.2 Phase A: PD Settling (0–1.5 s)

9.2.1 Goal

Bring the robot from its initial (possibly sprawled) configuration to a nominal standing posture with stable contact on all four feet.

9.2.2 Controller

A joint-space PD controller tracks nominal standing angles:

$$\boldsymbol{\tau}_{\mathrm{PD}} = \mathbf{K}_P\,(\boldsymbol{q}_{\mathrm{des}} - \boldsymbol{q}) - \mathbf{K}_D\,\dot{\boldsymbol{q}} \tag{9.1}$$

with $\boldsymbol{q}_{\mathrm{des}} = [0, 0.7, -1.4]$ rad per leg and $\mathbf{K}_P = \mathbf{K}_D = \mathrm{diag}(80, 80, 80)\,\mathrm{N\,m/rad}$, $\mathbf{K}_D = \mathrm{diag}(8, 8, 8)\,\mathrm{N\,m\,s/rad}$.

9.2.3 Height Estimation

During the last 0.8 s of Phase A, an IIR filter tracks the settled CoM height:

$$p_{z,\mathrm{settled}} \leftarrow 0.98\,p_{z,\mathrm{settled}} + 0.02\,p_z \tag{9.2}$$

This settled height ($\approx 0.338\,\mathrm{m}$) is used as $p_{z,\mathrm{des}}$ for all subsequent phases.

9.3 Phase B: All-Stance MPC (1.5–3.5 s)

9.3.1 Goal

Smoothly transition from PD joint-space control to GRF-based force control while maintaining full 4-foot contact. This "warm-starts" the MPC with a tractable (all-stance) problem before introducing the complexity of leg swing.

9.3.2 Contact Schedule

All 4 feet in stance for the full horizon:

$$\mathbf{S}[k, i] = 1 \quad \forall k, i \tag{9.3}$$

9.3.3 GRF Initialisation

Seeded at the static equilibrium:

$$f_{z,i}^{(0)} = \frac{mg}{4} = \frac{15.206 \times 9.81}{4} \approx 37.3\,\mathrm{N} \tag{9.4}$$

9.3.4 Phase B Warmup (100 ms)

During the first 100 ms of Phase B, the MPC runs at the physics step rate (2 ms) to rapidly build correct GRF estimates from the fresh measurement state. After warmup, the rate drops to the nominal 30 ms cadence.

9.4 Phase C: Trot MPC ($t > 3.5$ s)

9.4.1 Entry Gate

Phase C is entered only when:

1. $t > 3.5$ s (Phase B duration elapsed).
2. $p_z \geq 0.30$ m (robot is standing tall enough).
3. $|\theta| < 2°$ (body is level — prevents trot on tilted stance).

9.4.2 Phase C Warmup (300 ms)

After Phase C is entered, the gait scheduler is activated but all 4 feet remain in stance for 300 ms (1.5 gait cycles). This gives the MPC time to smooth forces before the first swing leg lifts off.

9.4.3 Normal Operation

After warmup:

1. Gait scheduler provides the $(K \times 4)$ contact schedule.
2. Lift-off transitions trigger `notify_lift()` on the swing controller (Raibert touchdown computed, trajectory initialised).
3. Stance legs: torques from MPC GRFs via $-\mathbf{J}^\top \boldsymbol{f}$.
4. Swing legs: torques from task-space PD via (8.12).
5. GRF solution is smoothed before application (Section 9.5).

9.5 GRF Smoothing

A first-order IIR filter blends consecutive QP solutions:

$$\boldsymbol{f}^{(t)} = \beta\, \boldsymbol{f}_{\text{new}} + (1-\beta)\, \boldsymbol{f}^{(t-1)} \tag{9.5}$$

$$\beta_{\text{flat}} = 0.75 \tag{9.6}$$

$$\beta_{\text{stairs}} = 0.65 \tag{9.7}$$

On stairs, a slower β prevents force spikes when contact geometry changes abruptly.

9.5.1 GRF Re-Seeding

When a leg transitions from swing to stance, its stored GRF is near zero. If it falls below 70% of the static equilibrium value, it is immediately re-seeded:

$$\text{if } f_{z,i} < 0.7\,\frac{mg}{n_{\text{stance}}} \quad \Rightarrow \quad \boldsymbol{f}_i \leftarrow \left[0,\ 0,\ \frac{mg}{n_{\text{stance}}}\right] \tag{9.8}$$

This eliminates the underforce window ($\approx$100 ms) that occurs when the smoothed GRF is too low on the first tick of stance.

9.6 Collapse Detection and Recovery

If $p_z < 0.20\,\text{m}$ while the MPC is active:

1. Switch to PD stand controller.
2. Hold PD for 2.0 s.
3. Return to Phase B (all-stance MPC).

Chapter 10

Flat-Ground Trotting: End-to-End Flow

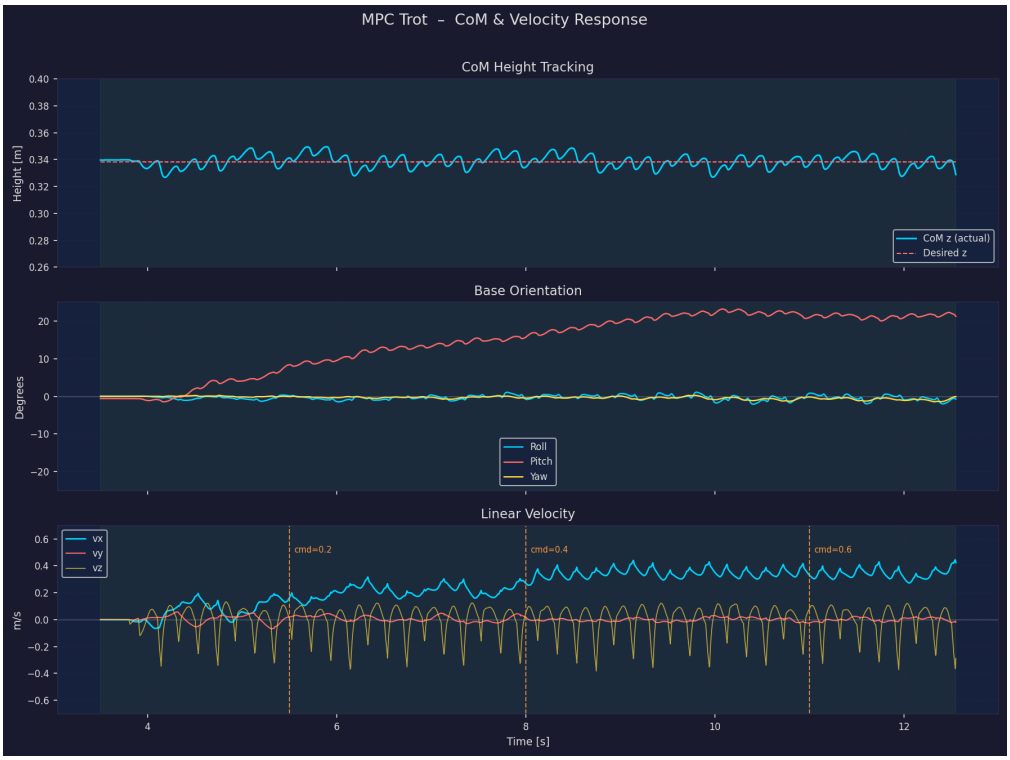
10.1 Step-by-Step Control Loop

Each physics step ($\Delta t_{\text{phys}} = 2\,\text{ms}$):

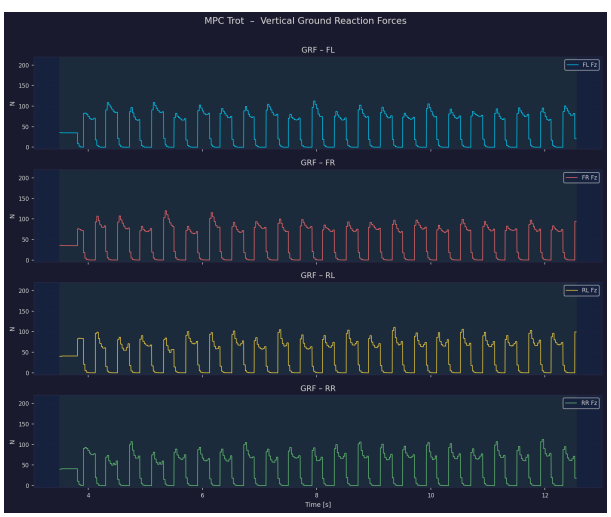
1. **State read.** `StateEstimator.update()` reads $\boldsymbol{x}$, joint positions, foot positions, contact forces.
2. **Contact detection.** Rolling-max buffer + height override (Eq. 5.3).
3. **Height integral.** $e_{z,\text{int}} \mathrel{+}= (p_{z,\text{des}} - p_z)\,\Delta t_{\text{phys}}$.
4. **Phase logic.** Select Phase A/B/C based on simulation time and height.
5. **MPC solve** (every 30 ms, rate-limited):
   (a) Build $\boldsymbol{x}_0 =$ `state.mpc_x()`.
   (b) Compute gait schedule $\mathbf{S} \in \{0,1\}^{K\times 4}$.
   (c) Build $\mathbf{A}_c, \mathbf{B}_c$ for each horizon step k.
   (d) Discretise: $(\mathbf{A}_d^{(k)}, \mathbf{B}_d^{(k)}) = \text{ZOH}(\mathbf{A}_c^{(k)}, \mathbf{B}_c^{(k)}, \Delta t_{\text{mpc}})$.
   (e) Condense: $(\mathbf{A}_{\text{qp}}, \mathbf{B}_{\text{qp}})$.
   (f) Build reference $\mathbf{X}_{\text{ref}}$.
   (g) Compute $\mathbf{H}, \boldsymbol{g}$ (cost).
   (h) Build constraints $\mathbf{C}, \boldsymbol{lb}, \boldsymbol{ub}$.
   (i) Solve QP $\Rightarrow \mathbf{U}_{\text{opt}}$.
   (j) Extract $\boldsymbol{f}_{1:4} = \mathbf{U}_{\text{opt}}[0{:}12]$.
   (k) Apply GRF smoothing.
6. **Swing detection.** Check lift-off transitions (ϕ_i crossed duty boundary); call `notify_lift()` with Raibert touchdown target.
7. **Torque computation.**
   - Stance: $\boldsymbol{\tau}_i = -\mathbf{J}_i^\top \boldsymbol{f}_i + \boldsymbol{\tau}_{\text{bias,i}}$.
   - Swing: $\boldsymbol{\tau}_i = \mathbf{J}_i^\top \boldsymbol{F}_{\text{task,i}} + \boldsymbol{\tau}_{\text{bias,i}}$.
8. **Actuator remapping.** Internal $\rightarrow$ MuJoCo order.
9. **Torque limit.** Clip to $\pm 33.5\,\text{N}\,\text{m}$ per joint.
10. **Apply.** `data.ctrl[:] = ctrl`; `mj_step()`.
11. **Log.** Write row to CSV debug log.

10.2 Results for Flat-Ground Trotting

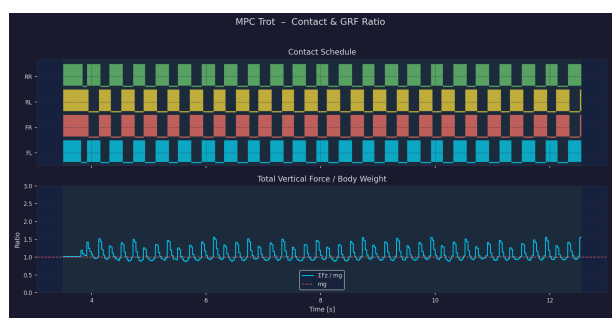
Below are the controller responses for trotting on flat ground



(a) COM, orentation and Velocity responses



(b) Ground Reaction Forces



(c) Contact Schedules

Figure 10.1: Controller responses during trotting on flat ground

Below is the video demonstration of the quadruped robot performing a trot gait on flat ground in the MuJoCo simulation environment: Watch Video Demonstration

Chapter 11

Stair Climbing System

11.1 Stair Scene Geometry

The `scene_stairs_uniform.xml` scene contains:

Table 11.1: Stair scene geometry

Section	**x range**	**Surface** z	**Steps**
Flat approach	$x < 2.00\,\mathrm{m}$	$0.00\,\mathrm{m}$	—
Ascending stairs	$2.00 \leq x < 3.40\,\mathrm{m}$	$0.06 \to 0.30\,\mathrm{m}$	5
Top platform	$3.40 \leq x < 4.40\,\mathrm{m}$	$0.30\,\mathrm{m}$	—
Descending stairs	$4.40 \leq x < 5.80\,\mathrm{m}$	$0.24 \to 0.06\,\mathrm{m}$	5
Flat exit	$x \geq 5.80\,\mathrm{m}$	$0.00\,\mathrm{m}$	—

Step parameters:

$$\begin{aligned} \text{Riser height:} \quad & h_r = 0.06\,\mathrm{m} \\ \text{Tread width:} \quad & w_t = 0.28\,\mathrm{m} \\ \text{Total height:} \quad & H = 5 \times 0.06 = 0.30\,\mathrm{m} \end{aligned}$$

11.2 Terrain Mode Activation

The controller initialises in "flat" mode (terrain estimator is a no-op). The simulation loop calls `switch_to_terrain_mode("st` when the robot's x-position exceeds the approach threshold ($x > 1.80\,\mathrm{m}$, i.e. $0.20\,\mathrm{m}$ before the first riser).

At the switch:

1. Terrain estimator mode set to "stairs".
2. Nominal clearance anchored: $\ell = p_{z,\text{settled}} - z_{\text{terrain}}$.
3. Stairs-specific Q weights activated immediately.
4. Swing clearance changes: $0.08\,\mathrm{m} \to 0.18\,\mathrm{m}$ (adaptive).

11.3 Terrain Height Map

11.3.1 Discrete Step Function

Used for touchdown z initialisation and failure detection:

$$z_{\text{terrain}}(x) = \begin{cases} 0 & x < 2.00 \\ \lceil \frac{x-2.00}{0.28} \rceil \times 0.06 & 2.00 \le x < 3.40 \quad \text{(ascending)} \\ 0.30 & 3.40 \le x < 4.40 \quad \text{(platform)} \\ 0.30 - \lceil \frac{x-4.40}{0.28} \rceil \times 0.06 & 4.40 \le x < 5.80 \quad \text{(descending)} \\ 0 & x \ge 5.80 \end{cases} \tag{11.1}$$

11.3.2 Smooth Ramp Approximation

Used for pitch_ref and target_pz to ensure smooth reference transitions:

An 8 cm linear pre-ramp before each riser edge blends the height continuously, preventing step-changes in the MPC reference.

11.4 Adaptive CoM Height

11.4.1 Target Height

$$p_{z,\text{target}}(x, \ell) = \underbrace{\frac{z_{\text{smooth}}(x + d_{\text{hip}}) + z_{\text{smooth}}(x - d_{\text{hip}})}{2}}_{\text{avg terrain under both hips}} + \ell \tag{11.2}$$

where $d_{\text{hip}} = 0.1934\,\text{m}$ and $\ell \approx 0.338\,\text{m}$ (nominal clearance).

11.4.2 Asymmetric Rate Limiter

$$\Delta p_{z,\text{des}} = \text{clip}(p_{z,\text{target}} - p_{z,\text{des}},\ -r_{\text{down}}\,\Delta t,\ r_{\text{up}}\,\Delta t) \tag{11.3}$$

$$r_{\text{up}} = 0.12\,\text{m/s} \quad \text{(ascending — fast to prevent lag)} \tag{11.4}$$

$$r_{\text{down}} = 0.04\,\text{m/s} \quad \text{(descending — slow to prevent drops)} \tag{11.5}$$

This means a 6 cm riser is tracked in $\approx 250\,\text{ms}$ on ascent (vs. 1.5 s at the original 0.04 m/s symmetric rate which caused sustained overforce and roll crash).

11.5 Body Pitch Reference

11.5.1 Geometric Derivation

The desired body pitch angle is computed from the terrain slope under the hip span:

$$\theta_{\text{ref}} = \text{clip}\left(\arctan\left(\frac{z_{\text{smooth}}(x + d_{\text{hip}}) - z_{\text{smooth}}(x - d_{\text{hip}})}{2\,d_{\text{hip}}}\right),\ -0.21, 0.21\right) \tag{11.6}$$

- Ascending ($2.00 \to 3.40\,\text{m}$): $\theta_{\text{ref}} \approx +0.154\,\text{rad} \approx +8.8^\circ$ (nose-up).
- Platform ($3.40 \to 4.40\,\text{m}$): $\theta_{\text{ref}} = 0$.
- Descending ($4.40 \to 5.80\,\text{m}$): $\theta_{\text{ref}} \approx -0.154\,\text{rad} \approx -8.8^\circ$ (nose-down).

Physical reason for negative pitch on descent: When front feet are on lower steps and rear feet are still higher, a level body shifts the CoM forward of the support polygon, creating a nose-dive torque. By commanding -8.8° pitch reference, the MPC proactively distributes forces to maintain this geometry throughout descent.

Flat approach guard: The smooth height function has an 8 cm pre-ramp before $x = 2.00\,\text{m}$. This would generate a phantom $+8^\circ$ pitch command on flat ground. The guard returns $\theta_{\text{ref}} = 0$ when $x < 2.00\,\text{m}$.

11.5.2 IIR Smoothing

The geometric pitch reference is filtered before passing to the MPC:

$$\theta_{\text{ref,smooth}} \leftarrow 0.3\,\theta_{\text{ref,smooth}} + 0.7\,\theta_{\text{ref,raw}} \tag{11.7}$$

The fast smoothing constant ($\alpha = 0.3$, settling in ≈ 2 ticks) prevents the MPC from generating anti-pitch torques during the 3–6° lag that occurred with $\alpha = 0.5$.

11.6 Per-Foot Stair Touchdown Placement

11.6.1 Tread Snapping

The Raibert heuristic computes a continuous touchdown x-position. On stairs, this must be clamped to a safe zone within the target tread (margin: 4 cm from each riser edge):

$$x_{\text{td,safe}} = \text{clip}(x_{\text{Raibert}},\ x_{\text{tread,lo}} + \delta,\ x_{\text{tread,hi}} - \delta)\,, \quad \delta = 0.04\,\text{m} \tag{11.8}$$

11.6.2 Height Map Touchdown

The touchdown z-coordinate is set from the deterministic height map:

$$z_{\text{td}} = z_{\text{terrain}}(x_{\text{td,safe}}) + r_{\text{foot}} \tag{11.9}$$

where $r_{\text{foot}} = 0.022\,\text{m}$ is the foot sphere radius.

This ensures the foot targets the correct tread surface even if the body has not yet risen to track the terrain.

11.7 Adaptive Swing Clearance

$$h_{\text{sw}}(z_{\text{terrain}}) = h_{\text{flat}} + \min\left(1,\ \frac{z_{\text{terrain}}}{z_{\text{ramp}}}\right) \times (h_{\text{terrain}} - h_{\text{flat}} - 0.01) + 0.01 \tag{11.10}$$

$$h_{\text{flat}} = 0.08\,\text{m}, \quad h_{\text{terrain}} = 0.18\,\text{m}, \quad z_{\text{ramp}} = 0.10\,\text{m}$$

As the terrain IIR estimate rises from $0 \to 0.30\,\text{m}$, swing clearance grows from $0.08 \to 0.18\,\text{m}$, ensuring the foot clears the step it is swinging to land on.

11.8 Early Touchdown Detection

When a swing leg physically contacts the terrain before its scheduled touchdown (e.g. the foot catches the riser face), the controller must stop pushing it forward and freeze it in place:

$$\text{early_td}[i] = \mathbf{1}\left[c_{f,i} > 10\,\text{N} \ \wedge\ \frac{t - t_{\text{liftoff}}}{T_{\text{swing}}} > 0.60\right] \tag{11.11}$$

The two guards in Eq. (11.11) are essential:

1. **Guard 1 (terrain mode only):** In flat mode, early touchdown is disabled. The rolling-max buffer retains stance contact forces for $\approx 20\,\text{ms}$ post-liftoff, causing false triggers.
2. **Guard 2 (60% swing progress):** Feet hitting the *riser face* at 20–30% swing progress would freeze far short of the safe tread zone. Requiring 60% ensures the foot has crested the riser and is descending toward the tread before early touchdown fires.

When early touchdown is active:

$$\boldsymbol{\tau}_i = \mathbf{J}_i^\top\left[\mathbf{K}_d\left(-\boldsymbol{v}_{\text{foot}}\right)\right] + \boldsymbol{\tau}_{\text{bias,i}} \quad \text{(velocity damping only, no position error)} \tag{11.12}$$

11.9 Support Plane Estimation

11.9.1 Purpose

When stance feet are on different stair steps, they are not coplanar. The support plane provides a best-fit terrain normal for rotating friction-cone constraints.

11.9.2 Least-Squares Plane Fit

Given $n \geq 3$ stance foot positions $\{\boldsymbol{p}_i\}$:

1. Compute centroid: $\boldsymbol{c} = \frac{1}{n}\sum_i \boldsymbol{p}_i$.
2. Form $\mathbf{A} = [\boldsymbol{p}_1 - \boldsymbol{c},\ \ldots,\ \boldsymbol{p}_n - \boldsymbol{c}]^\top$.
3. Compute SVD: $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$.
4. Normal: $\hat{\boldsymbol{n}} = \mathbf{V}[-1, :]$ (last row of $\mathbf{V}^\top$, smallest singular value direction).
5. Flip if $n_z < 0$.

11.9.3 Support Frame Construction

$$\hat{\boldsymbol{n}} = \hat{\boldsymbol{n}}/\|\hat{\boldsymbol{n}}\| \tag{11.13}$$

$$\hat{\boldsymbol{x}}_{\text{plane}} = \hat{\boldsymbol{x}}_{\text{world}} - (\hat{\boldsymbol{x}}_{\text{world}} \cdot \hat{\boldsymbol{n}})\,\hat{\boldsymbol{n}} \quad \text{(project world-}x\text{ onto plane)} \tag{11.14}$$

$$\hat{\boldsymbol{y}}_{\text{plane}} = \hat{\boldsymbol{n}} \times \hat{\boldsymbol{x}}_{\text{plane}} \tag{11.15}$$

$$\mathbf{R}_{\text{plane}} = [\hat{\boldsymbol{x}}_{\text{plane}},\ \hat{\boldsymbol{y}}_{\text{plane}},\ \hat{\boldsymbol{n}}] \tag{11.16}$$

$\mathbf{R}_{\text{plane}}$ is passed to the QP constraints for friction-cone rotation but *not* used in the $\mathbf{B}$ matrix (moment arms stay in world frame).

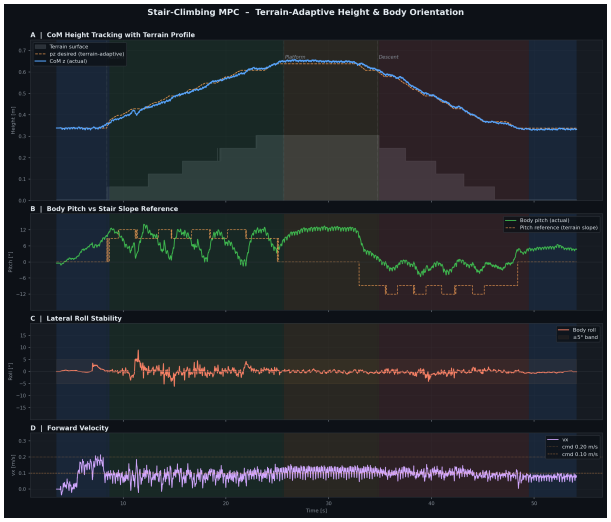
11.10 IIR Terrain Tracking

The terrain estimator maintains an IIR-filtered estimate of the terrain height from stance foot z-positions:

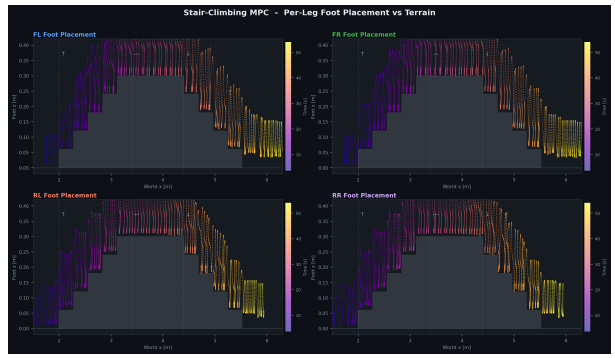
$$z_{\text{terrain}} \leftarrow 0.70\, z_{\text{terrain}} + 0.30\, \max(0,\ \overline{z}_{\text{stance}} - r_{\text{foot}}) \tag{11.17}$$

where $\overline{z}_{\text{stance}}$ is the mean z-position of all feet with $c_{f,i} > 10\,\text{N}$. This estimate drives the adaptive swing clearance and is used as the "current terrain" baseline for touchdown offset.

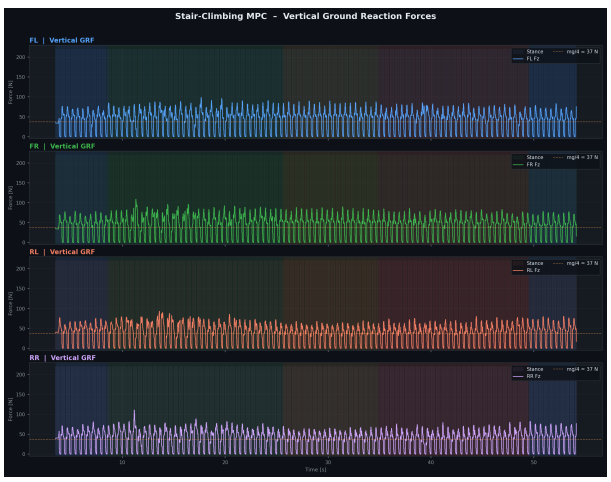
11.11 Results for Stairs Climbing



(a) Base Height and Orientation



(b) Foot Placement



(c) Ground Reaction Forces

(d) Contact Schedules

Figure 11.1: Controller responses during stair climbing

Below are the video demonstrations of the quadruped robot performing stair climbing and stair descending:

- **Stair Climbing Video:**
  Watch Climbing Demonstration
- **Stair Descending Video:**
  Watch Descending Demonstration

Chapter 12

Complete Constraints Reference

Table 12.1: All constraints active in the QP and elsewhere

Constraint	**Equation**	**Purpose / Notes**
Normal force min	$\hat{\boldsymbol{n}}^\top \boldsymbol{f}_i \geq 5\,\mathrm{N}$	Prevent foot from losing contact; ensure minimum support
Normal force max	$\hat{\boldsymbol{n}}^\top \boldsymbol{f}_i \leq 200\,\mathrm{N}$	Leg structural limit
Friction cone $+x$	$\hat{\boldsymbol{t}}_x^\top \boldsymbol{f}_i \leq \mu\, \hat{\boldsymbol{n}}^\top \boldsymbol{f}_i$	Linearised pyramid, positive tangential
Friction cone $-x$	$-\hat{\boldsymbol{t}}_x^\top \boldsymbol{f}_i \leq \mu\, \hat{\boldsymbol{n}}^\top \boldsymbol{f}_i$	Linearised pyramid, negative tangential
Friction cone $+y$	$\hat{\boldsymbol{t}}_y^\top \boldsymbol{f}_i \leq \mu\, \hat{\boldsymbol{n}}^\top \boldsymbol{f}_i$	Linearised pyramid, positive lateral
Friction cone $-y$	$-\hat{\boldsymbol{t}}_y^\top \boldsymbol{f}_i \leq \mu\, \hat{\boldsymbol{n}}^\top \boldsymbol{f}_i$	Linearised pyramid, negative lateral
Swing zero force	$\boldsymbol{f}_i = \boldsymbol{0}$	Swing leg cannot exert force (3 equality constraints)
Torque limit	$\lvert\tau_j\rvert \leq 33.5\,\mathrm{N\,m}$	Hardware safety limit, applied after all torque computation
Collapse gate	$p_z \geq 0.20\,\mathrm{m}$	Trigger PD recovery if violated
Trot gate	$p_z \geq 0.30\,\mathrm{m},\ \lvert\theta\rvert < 2^\circ$	Phase C entry condition
pz rate limit up	$\dot{p}_{z,\mathrm{des}} \leq 0.12\,\mathrm{m/s}$	Smooth height increase on stairs
pz rate limit down	$\dot{p}_{z,\mathrm{des}} \geq -0.04\,\mathrm{m/s}$	Smooth height decrease on stairs
Contact threshold	$c_{f,i} > 10\,\mathrm{N}$	Minimum force to classify as stance (for early touchdown and state estimator)
Early touchdown	$\phi_{\mathrm{swing}} > 0.60$	Guard against premature early touchdown detection
Tread margin	$x \in [x_{\mathrm{tread}} + 0.04,\ x_{\mathrm{tread+1}} - 0.04]$	Safe foot placement within each step
Pitch clip	$\lvert\theta_{\mathrm{ref}}\rvert \leq 0.21\,\mathrm{rad}$	Cap pitch reference at $\approx 12^\circ$

Chapter 13

Module Dependency and Data Flow

13.1 Module Dependency Graph

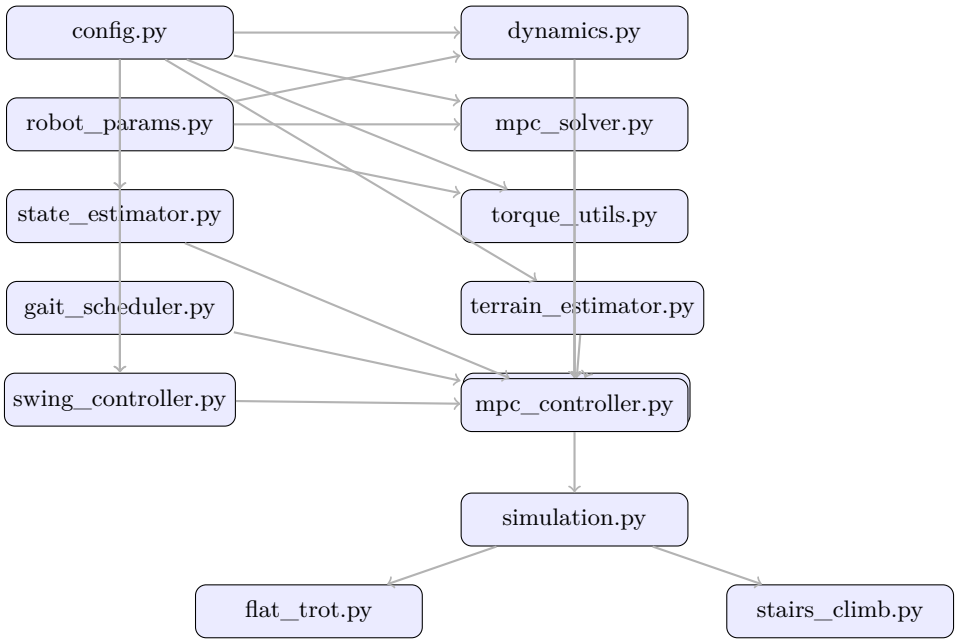


Figure 13.1: Module dependency graph

13.2 Per-Step Data Flow (Phase C)

1. **MuJoCo step** → **StateEstimator**: raw sensor data.
2. **StateEstimator** → **RobotState**: structured state.
3. **TerrainEstimator.update(state)**: IIR terrain height.
4. **TrotGait.schedule(t, dt, K)**: $(K \times 4)$ contact bool.
5. **TrotGait.contact_at(t)** → lift-off check: if transition detected, **SwingController.notify_lift()**.
6. **RigidBodyDynamics.Ac/Bc** for each k → **ZOH** → $(\mathbf{A}_d^{(k)}, \mathbf{B}_d^{(k)})$.
7. **ConvexMPC.condense** → $(\mathbf{A}_{\text{qp}}, \mathbf{B}_{\text{qp}})$.

8. **make_reference** $\rightarrow \mathbf{X}_{\text{ref}}$.
9. **ConvexMPC.cost** $\rightarrow (\mathbf{H}, \boldsymbol{g})$.
10. **ConvexMPC.constraints** $\rightarrow (\mathbf{C}, \boldsymbol{lb}, \boldsymbol{ub})$.
11. **ConvexMPC.solve** $\rightarrow \mathbf{U}_{\text{opt}}$.
12. GRF smooth + re-seed $\rightarrow \boldsymbol{f}_{1:4}$.
13. **grf_to_joint_torques**: $\boldsymbol{\tau}_{\text{stance}}$.
14. **SwingController.compute_torques**: $\boldsymbol{\tau}_{\text{swing}}$.
15. Blend stance/swing torques by schedule.
16. **leg_to_ctrl** (reorder) $\rightarrow$ clip $\rightarrow$ `data.ctrl`.
17. **mj_step()**.

Chapter 14

Bug Fixes and Design Decisions

Table 14.1: Complete bug fix and design decision log (19 items)

#	Location	Symptom / Root Cause	Fix
1	torque_utils.py	Torque sign error: robot pushed itself into the ground. MPC gives GRF = upward force; joint must push *down*.	Changed $+\mathbf{J}^\top \boldsymbol{f}$ to $-\mathbf{J}^\top \boldsymbol{f}$
2	state_estimator.py	Phase A→B boundary: efc_force drops to 0 for ∼300 ms. Contact reports NNNN; MPC sees all-swing, generates 3× overforce.	Height-based override: force contacts=True when $p_z > 0.20\,\mathrm{m}$
3	config.py	Contact threshold $\tau_{\mathrm{cf}} = 1\,\mathrm{N}$ — below MuJoCo solver noise.	Raised to 10 N
4	mpc_controller.py	Phase C reset pz_des to NOMINAL_HEIGHT (0.32 m); body drops 18 mm before first swing.	Keep pz_des = settled height (0.338 m) throughout
5	gait_scheduler.py	Gait phase measured from $t = 0$; mid-cycle state at Phase C activation.	Anchor gait phase to activation time t_0 via `activate(t)`
6	mpc_controller.py	Q weights 10–50× too large (roll=100, pitch=200, pz=500); aggressive corrective forces destabilise trot.	Matched to CasADi reference values
7	swing_controller.py	Raibert overcorrection: $(t_{\mathrm{pred}} + k_v)\, v_{\mathrm{actual}}$ applied as one term; 3.5× too large.	Separated nominal drift $(v_{\mathrm{cmd}}\, t_{\mathrm{pred}})$ from error correction $(k_v\,(v_{\mathrm{actual}} - v_{\mathrm{cmd}}))$
8	mpc_controller.py	cmd_vx/vy in body frame; Raibert needs world frame.	Rotate by yaw: $[v_x^w, v_y^w]^\top = \mathbf{R}_z(\psi)[v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}]^\top$
9	swing_controller.py	Foot velocity used only 3-DOF leg Jacobian; ignored body rolling/pitching.	Use full $(3 \times n_v)$ Jacobian: $\boldsymbol{v}_{\mathrm{foot}} = \mathbf{J}_{\mathrm{full}}\, \dot{\boldsymbol{q}}$
10	dynamics.py	Moment arm rotated into support-plane frame while forces stayed in world frame; physically wrong cross product.	Keep $\boldsymbol{r}_i$ in world frame; rotate only friction cones

(continued)

#	Location	Symptom / Root Cause	Fix
11	mpc_solver.py	GRF smoothing was a no-op (result not stored back).	Fixed: `self._grf` $= \beta\, \boldsymbol{f}_{\text{new}} + (1 - \beta)$ `self._grf`
12	mpc_controller.py	Terrain mode not reset in `simulation.reset()`; stair mode bled into viewer run.	Added `terrain.set_mode("flat")` to `reset()`
13	swing_controller.py	Early touchdown fired on buffer bleed (20 ms post-liftoff) in flat mode.	Guard 1: early touchdown disabled in flat mode
14	swing_controller.py	Early touchdown at 20% swing progress froze foot on riser face, far short of tread.	Guard 2: require $>$ 60% swing progress before early touchdown
15	terrain_estimator.py	Smooth height pre-ramp generates phantom +8° pitch on flat approach.	Guard: return 0.0 when $x < 2.00$ m
16	mpc_controller.py	stairs_Q not activated until $x >$ 2.00 m; first step hit with base Q (py=1, vy=2).	Activate stairs_Q as soon as terrain mode switches
17	mpc_controller.py	pitch_ref IIR $\alpha = 0.5$ caused 3–6° lag; MPC generated anti-pitch torques opposing climb.	Changed to $\alpha = 0.3$ (70% new, 30% old)
18	mpc_controller.py	Symmetric pz rate limiter 0.05 m/s: 30 mm riser took 600 ms; sustained overforce + roll crash.	Asymmetric: 0.12 m/s up, 0.04 m/s down
19	mpc_controller.py	GRF seed threshold 50% insufficient; 100 ms underforce window on new stance foot.	Raised to 70% seed threshold

Chapter 15

Numerical Summary

Table 15.1: Key numerical parameters

Parameter	**Value**
Physics timestep	2 ms (500 Hz)
MPC solve rate	30 ms (33 Hz)
MPC horizon K	10 steps
Gait period T	0.40 s (2.5 Hz)
Duty cycle d	50%
Stance time	0.20 s
Swing time	0.20 s
State dimension n_s	13
Input dimension n_u	12 (4 feet $\times$ 3 forces)
QP size (stacked)	$H \in \mathbb{R}^{120\times 120}$
Regularisation α	3×10^{-4}
Friction coefficient μ	0.70
Foot normal force range	$[5, 200]$ N
Torque limit per joint	33.5 N m
Phase A duration	1.5 s
Phase B duration	2.0 s
Phase C warmup	0.30 s
Nominal CoM height	0.32 m
Settled CoM height	≈ 0.338 m
Collapse threshold	0.20 m
Trot ready threshold	0.30 m
Contact force threshold	10 N
GRF blend (flat)	$\beta = 0.75$
GRF blend (stairs)	$\beta = 0.65$
Stair approach x	1.80 m
Flat Raibert $k_{v,x}$	$0.4T = 0.16$ m s
Flat Raibert $k_{v,y}$	$0.05T = 0.02$ m s
Swing clearance (flat)	0.08 m
Swing clearance (stairs)	$0.08 \rightarrow 0.18$ m (adaptive)
pz rate up	0.12 m/s
pz rate down	0.04 m/s
Pitch IIR α	0.30 (70% new)
Tread safety margin	0.04 m
Early touchdown guard	$>$ 60% swing progress

Chapter 16

Conclusions

16.1 Summary

This project implements a complete convex MPC trot controller for the Unitree Go2 quadruped robot in MuJoCo simulation. The controller:

1. Formulates the stance-force problem as a convex QP (13 states, 12 inputs, $K = 10$ horizon) solvable in real time.
2. Uses a three-phase state machine (PD settle $\rightarrow$ all-stance MPC $\rightarrow$ trot MPC) to ensure smooth transitions.
3. Implements a minimum-jerk swing trajectory with Raibert touchdown placement for forward progression.
4. Extends to stair climbing through terrain-aware references (adaptive p_z, pitch reference), per-foot stair snapping, adaptive swing clearance, support-plane friction cones, and early touchdown detection.
5. Documents 19 bug fixes across 4+ debugging iterations, covering signal processing (contact detection), control theory (Raibert formulation, frame conventions), and numerical stability (GRF smoothing, rate limiters).

16.2 Limitations and Future Work

- The SRB model ignores leg mass ($\approx 10\%$ of total), which limits accuracy during dynamic swing. A whole-body MPC would improve this at significant computational cost.
- The trot gait uses a fixed period. Adaptive gait timing (speed-dependent T) would improve efficiency over a wider velocity range.
- The terrain estimator uses a hardcoded stair geometry. A general elevation map (e.g. from depth sensors) would enable uneven terrain.
- The friction coefficient $\mu = 0.7$ is fixed. Online friction estimation from force/slip measurements would improve robustness.
- The controller has been validated only in simulation. Sim-to-real transfer requires addressing actuator delays, sensor noise, and unmodelled flexibility.

Bibliography

[1] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, "Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 7440–7447, 2018.

[2] M. H. Raibert, *Legged Robots That Balance*. MIT Press, Cambridge, MA, 1986.

[3] Unitree Robotics, "Go2 Robot Dog," `https://www.unitree.com/go2/`, 2023.

[4] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ IROS*, pp. 5026–5033, 2012.